

tf.compat.v1.nn.embedding_lookup Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/embedding_lookup

Looks up embeddings for the given ids from a list of tensors.

Function Signatures

```
tf.compat.v1.nn.embedding_lookup( params, ids,  
partition_strategy='mod', name=None, validate_indices=True,  
max_norm=None )
```

Code Examples

```
tf.compat.v1.nn.embedding_lookup(  
    params,  
    ids,  
    partition_strategy='mod',  
    name=None,  
    validate_indices=True,  
    max_norm=None  
)
```

```
tf.compat.v1.nn.embedding_lookup(  
    params,  
    ids,  
    partition_strategy='mod',  
    name=None,  
    validate_indices=True,  
    max_norm=None  
)
```

Documentation

Looks up embeddings for the given ids from a list of tensors.

This function is used to perform parallel lookups on the list of tensors in `params`. It is a generalization of `tf.gather`, where `params` is interpreted as a partitioning of a large embedding tensor. `params` may be a `PartitionedVariable` as returned by using `tf.compat.v1.get_variable()` with a partitioner.

If `len(params) > 1`, each element `id` of `ids` is partitioned between the elements of `params` according to the `partition_strategy`.

In all strategies, if the `id` space does not evenly divide the number of

partitions, each of the first $(\text{max_id} + 1) \% \text{len}(\text{params})$ partitions will be assigned one more id.

If `partition_strategy` is "mod", we assign each id to partition

$p = \text{id} \% \text{len}(\text{params})$. For instance,

13 ids are split across 5 partitions as:

`[[0, 5, 10], [1, 6, 11], [2, 7, 12], [3, 8], [4, 9]]`

If `partition_strategy` is "div", we assign ids to partitions in a

contiguous manner. In this case, 13 ids are split across 5 partitions as:

`[[0, 1, 2], [3, 4, 5], [6, 7, 8], [9, 10], [11, 12]]`

If the input ids are ragged tensors, partition variables are not supported and the partition strategy and the `max_norm` are ignored.

The results of the lookup are concatenated into a dense

tensor. The returned tensor has shape `shape(ids) + shape(params)[1:]`.

Returns

Raises